

# Repaso Final Parcial 3

---

## Implementación y aceptación del software

Este proceso tiene como objetivo la entrega y aceptación del software en su totalidad y la realización de todas las actividades necesarias para la aceptación del mismo.

Fases para la implementación de software:

1. Preparar un ambiente operacional y uno de pruebas separados
  - Ambiente o plataforma: es la combinación específica de hardware y software que nos permite correr un software.
  - Ambiente operacional: la plataforma donde corre el sistema actual
  - Ambiente de prueba: la plataforma utilizada para desarrollar y dar mantenimiento a los sistemas.
2. Ofrecer capacitación a los usuarios, administradores y técnicos
  - Se les debe ofrecer a los usuarios términos o palabras clave
  - Una guía para sacar adelante los problemas que se presenten
  - Una lista de preguntas frecuentes
  - A los administradores, toda la documentación, y toda la parte de reportes
3. Realizar la conversión de datos y el cambio de sistema
  - Consiste en, cargar el nuevo sistema y los datos existentes. Dependiendo del sistema puede hacerse antes, durante o después de completar el ambiente operacional.
  - El proceso de cambio del sistema consiste en poner en línea el nuevo sistema y en retirar el anterior. Puede realizarse de forma directa, en paralelo, mediante etapas, dependiendo del riesgo implícito y del tiempo disponible para realizar la tarea.
  - Tipos de conversión de datos
4. Efectuar una evaluación luego de la instalación del sistema
  - Una vez instalado el sistema, se pone énfasis en determinar si el software cumple ciertos requisitos, permite lograr los objetivos del usuario y produce beneficios para los cuales fue aprobado.
5. Presentar un reporte final a la administración
  - El informe final debe incluir: documentación final y completa de todo el software, modificaciones o mejoras a realizar a futuro que fueron detectadas, recapitulación de

los presupuestos y cronogramas utilizados durante la instalación y los resultados de los test correspondientes a la evaluación final.

## ¿Qué es el mantenimiento de software?

Es el proceso de modificar un software después de que ha sido entregado y puesto en producción, con el objetivo de corregir errores, mejorar el rendimiento, adaptarlo a cambios en el entorno o añadir nuevas funcionalidades. Es una de las etapas más largas y costosas del ciclo de vida del software. Se estima que el 60-80% del costo total del software corresponde a su mantenimiento.

### Tipos de mantenimiento

1. **Mantenimiento correctivo:** corregir errores detectados después de la entrega del software. Cuando algún fallo o error ya sucedió, entonces se actúa para solucionarlo.
2. **Mantenimiento adaptativo:** ajustar el software a cambios del entorno externo o del hardware. Se adapta a cambios, por ejemplo se cambia el sistema operativo de la empresa, se hace una migración a la nube o hay cambios en la base de datos.
3. **Mantenimiento perfectivo:** mejorar el rendimiento o la mantenibilidad del software, o añadir nuevas funcionalidades solicitadas por el usuario. Prácticamente se refiere a la mejora continua. <-
4. **Mantenimiento preventivo:** prevenir posibles errores futuros o hacer el software más robusto. Por ejemplo, refactorización de código, actualización de librerías obsoletas.

## Service Mesh

Es una capa de infraestructura de red dedicada que se encarga de gestionar las comunicaciones entre microservicios, sin que estos tengan que incluir esa lógica en su código. En una arquitectura de microservicios, cada servicio se comunica con otros a través de llamadas HTTP/gRPC. Sin un **Service Mesh**, todo lo relacionado con seguridad, reintentos, enrutamientos, logs de tráfico, entre otros, debe implementarse dentro de cada microservicio. Con **Service Mesh**, todo esto se saca del código de los microservicios y se gestiona desde una capa externa, mediante un conjunto de proxies ligeros y un plano de control centralizado.

### Principales Ventajas

1. Observabilidad
  1. Métricas, latencia, tasas de error, número de peticiones por segundo, entre otras
  2. Tracing distribuido, seguir una solicitud a lo largo de múltiples servicios para detectar cuellos de botella
2. Gestión de tráfico
  1. Balanceo de carga inteligente

2. Timeout configurable y circuit breakers
3. Failover, redirigir automáticamente si un servicio falla  
Todo esto lo hace el *Sidecar Proxy*, según las reglas del control plane

### 3. Seguridad

1. Autenticación mutua y cifrado por medio de TLS
2. Control de acceso, definir qué servicios pueden hablar con cuáles
3. Cifrado de tráfico en tránsito

Todo el tráfico puede estar cifrado y controlado por políticas detalladas sin modificar el código de los microservicios

## ¿Cómo funciona?

1. Sidecar Proxy (Plano de datos / Data Plane), cada microservicio tiene asociado un Proxy (Envoy, es un proxy de código abierto) que corre como un contenedor adicional en el mismo contenedor, a este patrón se le llama *Sidecar*

El Data Plane es la parte que maneja directamente el tráfico de red entre servicios.

- Todo el tráfico entre y sale a través de ese proxy, no directamente al servicio.
- Así el proxy, puede aplicar reglas como: "todo el tráfico hacia este servicio debe ir cifrado" o "si este servicio falla, reintenta 3 veces mas antes de marcarlo como caído"

2. Control Plane (Plano de control), es el cerebro del Service Mesh. Por ejemplo: Istio (Istiod), Linkerd control plane.

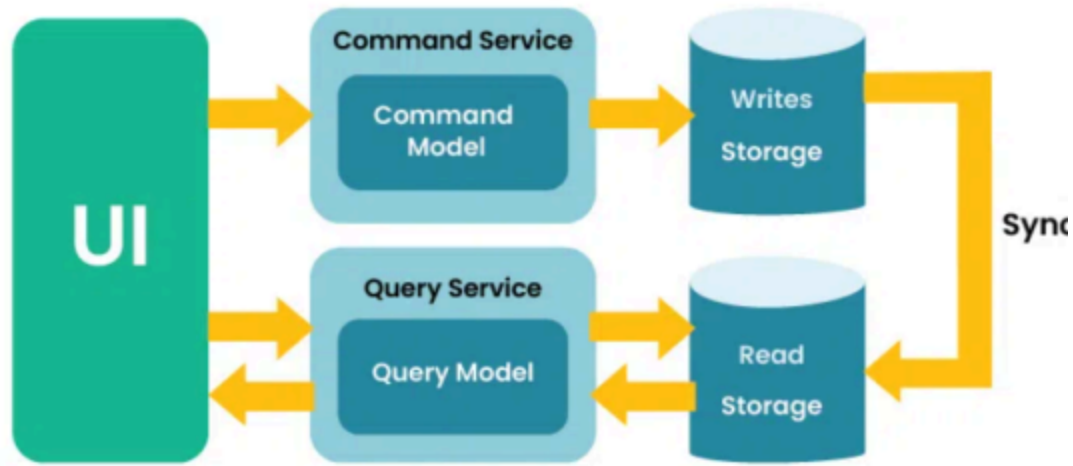
- Configura y gestiona los proxies sidecar
- Aplica las políticas de seguridad, enrutamiento, control de acceso, balanceo, entre otras
- Recolecta métricas, logs y trazas distribuidas desde los proxies.

El *Control Plane* es la parte que configura, coordina y supervisa todo el comportamiento del mesh.

## CQRS

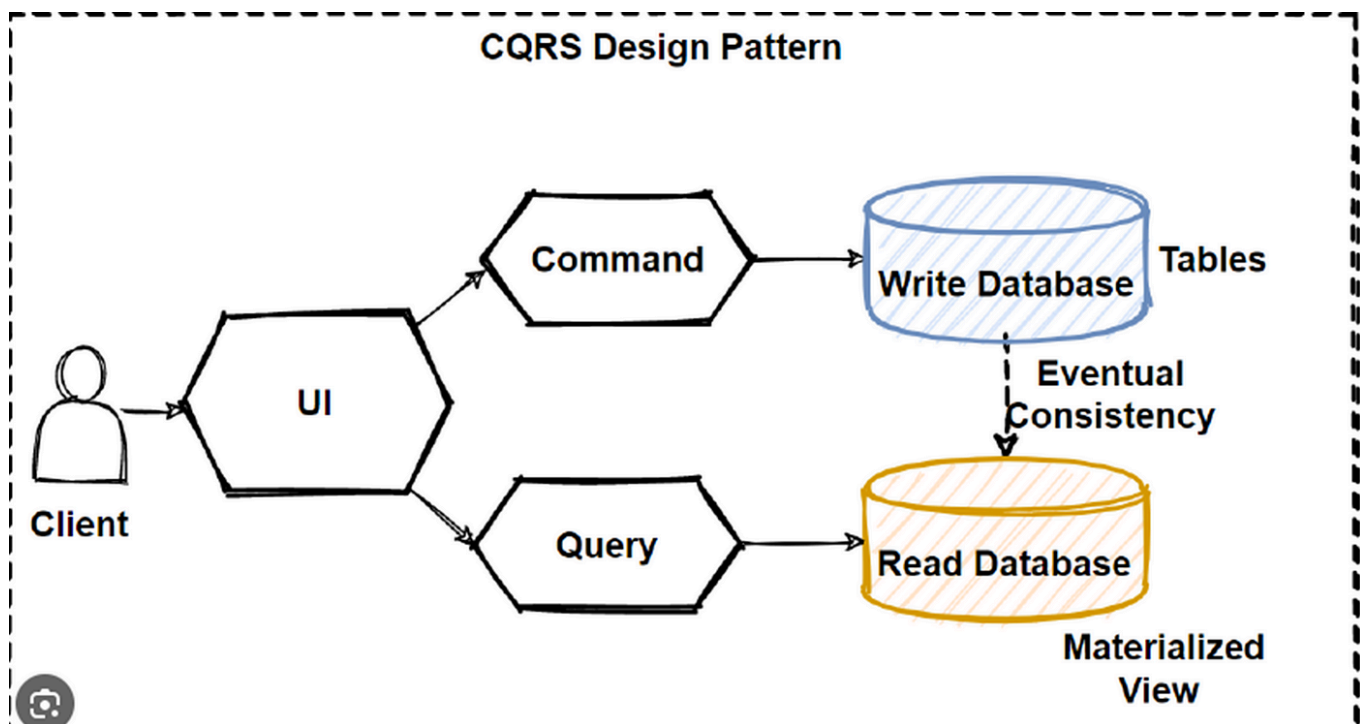
### Command Query Responsibility Segregation

Es un patrón de arquitectura de software que separa las operaciones de lectura (queries: separación de datos) de las operaciones de escritura (commands: inserción, actualización, y borrado de datos).



### Modelo de escritura (Command Model)

- Se encarga de recibir y procesar comandos (acciones que modifican el estado del sistema)
- Contiene la lógica de negocio y las reglas de validación
- En arquitecturas mas avanzadas, puede estar respaldado por "*Event Sourcing*", es un patrón de diseño que registra todos los cambios que ocurren en un sistema como una secuencia de eventos. Es decir, se crea un historial de todo lo que sucede en otra base de datos



### Modelo de lectura (Query Model)

- Se encarga de recuperar información de manera eficiente
- No tiene lógica de negocio, solo sirve para leer datos rápidamente

## ¿Cómo funciona?

### *Modelo de Escritura (Command Model):*

- *Command*, es un DTO (objeto que transporta datos entre procesos) con toda la información necesaria para realizar la escritura y realizar una acción en nuestro sistema que acabe modificando el estado.
- *Command Handler*, es un servicio que ejecuta las acciones necesarias para cumplir con la intención del usuario. Por ejemplo, separa los comandos en diferentes archivos, lo que facilita la edición y recarga de cada comando.
- *Command Bus*, encargado de trasladar el Command (DTO) a su Handler correspondiente

### *Modelo de Lectura (Query Model):*

- *Query*, DTO con toda la información necesaria para realizar la lectura sin alterar el estado.
- *Query Handler*, servicio encargado de obtener esa información y devolvérsela al controlador.
- *Query Bus*, encargado de trasladar el DTO a su Handler correspondiente.

## Preguntas

1. ¿Qué significa CQRS y cuál es su propósito?
  - Command Query Responsibility Segregation (Segregación de Responsabilidades de Consultas de Comandos), su propósito es dividir las operaciones de lectura (Query Model) de las operaciones de escritura (Command Model) en un software para mejorar el rendimiento, escalabilidad y seguridad
2. ¿Cómo se diferencia CQRS de una arquitectura tradicional?
  - La principal diferencia es que en una arquitectura tradicional las operaciones de escritura y lectura tienen el mismo punto de acceso o comparten la misma base de datos, en CQRS se dividen las operaciones de escritura y de lectura en distintas bases de datos
3. ¿Cuáles son los principales beneficios de CQRS?
  - Al dividir las operaciones de escritura y de lectura mejora el rendimiento y hace que la app sea más rápida para devolver datos. Como también permite más control y mayor seguridad sobre los procesos porque se pueden restringir accesos según el tipo de operación que el usuario realice
4. ¿Por qué CQRS se combina frecuentemente con Event Sourcing?
  - Event Sourcing es un patrón de diseño que registra todos los cambios en una aplicación como una secuencia de eventos, es decir, cada cambio es guardado en una especie de historial y este a la vez permite la reconstrucción del estado de los datos y permite un mayor control de todo lo que se realiza
5. ¿Qué tipo de sistemas se benefician más de CQRS?

- Aplicaciones con alta demanda de procesos de lectura y escritura como también aplicaciones donde se requiera un alto control e historial de todo proceso de forma detallada
6. ¿Cuáles son los componentes principales en una arquitectura CQRS?
    - Modelo de escrita (Command Model), Command, Command Handler, Command Bus
    - Modelo de lectura (Query Model), Query, Query Handler, Query Bus
  7. ¿Cómo se gestionan las consultas en CQRS?
    - Las consultas se realizan a través del Modelo de lectura (Query Model), esto porque lo que se busca es acceder rápidamente a la información
  8. ¿Cuál es el rol de los Commands en CQRS?
    - Son DTO (objeto que transporta datos entre procesos) que llegan al Command Handler que es donde se ejecuta la lógica del negocio y logra cambiar el estado del sistema
  9. ¿Qué desafíos puede representar la implementación de CQRS?
    - Implica mayores esfuerzos en términos de infraestructura y mantenimiento, y lo más importante es que se requiere sincronización entre el Command Model y el Query Model
  10. ¿Cuándo no se recomienda usar CQRS?
    - En aplicaciones pequeñas donde no se reciban muchas consultas, ya que separa operaciones de lectura y escritura solo van añadir complejidad al desarrollo y no va aportar mayor beneficio

## gRPC

gRPC (gRPC Remote Procedure Call, Llamada a procedimiento remoto), es un framework de comunicación remota desarrollado por Google que permite a las aplicaciones comunicarse entre sí de forma eficiente, rápida y segura, especialmente útil en arquitecturas distribuidas como microservicios.

*Se basa en llamadas a procedimientos remotos, esto significa que una aplicación puede invocar funciones en otra aplicación, aunque estén en servidores diferentes, como si fueran funciones locales.*

gRPC (Remote Procedure Call, Llamada a procedimiento remoto) se basa en:

- Protocol Buffers (Protobuf), un sistema de serialización de datos muy eficiente
- HTTP/2, un protocolo moderno que permite mayor rendimiento que HTTP/1.1

### ¿Por qué usar gRPC?

1. Rendimiento mucho mayor que REST y JSON
2. Contratos de servicio claro, se define con .proto

3. Lenguaje agnóstico, genera código cliente y servidor para múltiples lenguajes
4. ideal para microservicios, bajo consumo de recursos, alta velocidad
5. Soporte nativo para streaming, autenticación comprensión y más

### "¿Cómo funciona gRPC? (Componentes principales)"

1. Archivo ".proto"
  - Es el archivo donde se define el servicio, los métodos y los tipos de mensajes. Este archivo se compila usando protoc (es el compilador de Protocol Buffers), generando código tanto para cliente como para servidor
2. Cliente gRPC
  - Llama a métodos remotos como si fueran funciones locales
  - No se preocupa por detalles de red, serialización, entre otras
  - El código del cliente lo genera el compilador "protoc"
3. Servidor gRPC
  - Implementa los métodos definidos en el .proto
  - Escucha peticiones y responde usando Protobuf
  - También se genera base con protoc y luego se programa la lógica
4. Transporte HTTP/2

### Protocol Buffers (Protobuf)

Protobuf convierte los datos en una secuencia binaria compacta, lo cual reduce el uso de red y mejora el rendimiento

### gRPC y la seguridad

Usa TLS por defecto para cifrar la comunicación.

### Flujo de Funcionamiento

1. Se tiene que crear el archivo .proto, en este se define qué métodos o funciones existen y qué datos usan. Es como un "contrato"
  2. Se compila el archivo .proto con protoc, esto genera el código base para el servidor que es donde tengo la lógica y también genera el código base para el cliente que es cómo el cliente están conectados y cómo enviar datos
  3. El servidor ejecuta la función, es decir, realiza todo el proceso de la función que se desee
  4. El cliente invoca a la función, esto lo hace como si fuera de manera local pero en realidad la función se encuentra en el backend y gRPC la pasa por el protocolo HTTP/2 y así es como llega hasta el backend y se ejecuta
- Por medio del protocolo HTTP/2 viajan las peticiones y respuestas.



## Preguntas

- **¿Qué significa gRPC y cuál es su propósito principal?**  
gRPC significa "**Google Remote Procedure Call**" y es un framework de comunicación diseñado para permitir la comunicación eficiente entre sistemas distribuidos mediante llamadas a procedimientos remotos (RPC).
- **¿En qué tecnología se basa gRPC para la serialización de datos?**  
Se basa en **Protocol Buffers (protobuf)**, un formato de serialización binario eficiente y compacto desarrollado por Google.
- **¿Cuál es la principal diferencia entre HTTP/1.1 y HTTP/2 en el contexto de gRPC?**  
HTTP/2 permite **multiplexación**, lo que significa que varias solicitudes pueden enviarse en paralelo sin esperas, reduciendo la latencia y mejorando el rendimiento de gRPC en comparación con HTTP/1.1.
- **Menciona al menos tres características principales de gRPC.**  
Uso de **Protocol Buffers (protobuf)** para serialización de datos.  
Soporte para **streaming bidireccional** en tiempo real.  
**Compatibilidad con múltiples lenguajes** como Python, Java, C++, Go, entre otros.
- **¿Cuáles son los pasos clave en el funcionamiento de gRPC?**  
Definir el servicio en un archivo `.proto`.  
Generar el código para el cliente y el servidor en distintos lenguajes.  
Establecer comunicación a través de **HTTP/2**.  
Serializar los datos con **Protocol Buffers**.  
Implementar autenticación y seguridad con **TLS/SSL**.
- **¿Qué ventajas ofrece gRPC en términos de seguridad?**  
Soporta **TLS/SSL** para encriptar la comunicación.



Implementa **mTLS (mutual TLS)** para autenticación segura entre cliente y servidor.

Puede integrarse con sistemas de control de acceso y autenticación avanzada.

- **¿Cómo mejora gRPC el rendimiento en comparación con REST?**

Usa **Protocol Buffers** en lugar de JSON, lo que reduce el tamaño de los mensajes.

Funciona sobre **HTTP/2**, lo que permite multiplexación y mejora la latencia. Permite **streaming bidireccional**, optimizando el intercambio de datos en tiempo real.

- **¿Cuáles son los cuatro tipos de métodos de llamada en gRPC?**

**Llamada Unaria:** Un solo mensaje de solicitud y una respuesta.

**Streaming del Servidor:** El servidor envía múltiples respuestas a una sola solicitud.

**Streaming del Cliente:** El cliente envía múltiples solicitudes antes de recibir una respuesta.

**Streaming Bidireccional:** Tanto el cliente como el servidor intercambian múltiples mensajes en ambas direcciones.

- **Menciona al menos dos casos de uso donde gRPC es una mejor opción que REST.**

**Microservicios**, donde se requiere comunicación eficiente entre servicios.

**Sistemas en tiempo real**, como chat en vivo o transmisión de medios.

- **¿En qué situaciones se recomienda utilizar gRPC?**

- ✓ Cuando se necesita **alto rendimiento y baja latencia**.
- ✓ En arquitecturas de **microservicios** para comunicación eficiente.
- ✓ Para **transmisión de datos en tiempo real** (streaming).
- ✓ Si la aplicación debe soportar **múltiples lenguajes de programación**.
- ✓ Cuando se requiere **seguridad avanzada** con TLS/mTLS.

## Gestión de Servicios ITSM

*Gestión* utilizar y adaptar la tecnología de forma que aporte un valor real medible, incorporar TI a la estrategia y a la táctica del negocio, no solo a la operativa.

Son las siglas de Gestión de Servicios de TI (IT Service Management).

Es un enfoque estructurado para diseñar, entregar, gestionar y mejorar los servicios de TI que una organización ofrece a sus usuarios o clientes.

*Su objetivo principal* es alinear los servicios de TI con las necesidades del negocio, asegurando calidad, eficiencia, valor y mejora continua.

Principios básicos de ITSM

1. Enfoque en el servicio, no solo en la tecnología
2. Orientación al cliente, cumplir y superar expectativas
3. Procesos definidos, para estandarizar o automatizar procesos repetitivos y mejora
4. Mejora continua, siempre busca optimizar.

### *Marcos de trabajo de ITSM*

1. ITIL (Information Technology Infrastructure Library)
  - Es el más conocido y más usado en ITSM
  - Proporciona un conjunto de buenas prácticas para gestionar servicios de TI a lo largo de su ciclo de vida.
  - Se enfoca en:
    - Estrategia del servicio
    - Diseño del servicio
    - Transición del servicio
    - Operación del servicio
    - Mejora continua del servicio.
2. COBIT (Control Objectives for Information and Related Technologies) **Es un marco de Gobierno de TI**
  - Enfocado en Gobierno y Control de TI
  - Ayuda a asegurar que la TI está alineada con los objetivos del negocio.
  - Es útil para auditorías, cumplimiento y gestión de riesgos.
3. Six Sigma
  - No es exclusivo de TI, viene de manufactura
  - Se usa para mejorar procesos reduciendo defectos y variabilidad.
  - Se aplica en ITSM para:
    - Mejorar calidad del servicio
    - Analizar problemas repetitivos
    - Estandarizar procesos.
4. ISO/IEC 20000
  - Norma internacional certificable para ITSM
  - Similar a ITIL, pero más formal y regulada
  - Establece requisitos para un sistema de gestión de servicios de TI
  - Se puede auditar y certificar (a diferencia de ITIL, que es solo un marco de buenas prácticas)